第 10 章：任务自动化 Agent

版本: v2.5 (2026-03-23 全书完成) **作者**: 内容撰写专家（场景篇）
状态: review（待技术审核）
最后更新: 2026-03-23

本章涉及面试题

1. **如何将复杂任务分解为可执行的子任务？**（涉及 10.2 节）
2. **任务执行失败时有哪些处理策略？**（涉及 10.4 节）
3. **如何设计工具调用的错误处理和重试机制？**（涉及 10.3 节）
4. **定时任务如何调度和监控？如何避免重复执行？**（涉及 10.5 节）

本章概述

学习目标： - 理解任务自动化 Agent 的核心挑战：任务分解、工具集成、异常处理 - 掌握任务编排策略和工具调用机制 - 能够设计异常处理和重试机制 - 理解任务自动化 Agent 的最佳实践与常见陷阱

核心知识点： - 任务自动化场景分类与核心挑战 - 任务分解方法（层级分解、LLM 辅助、模板化） - 任务编排策略（串行/并行/条件分支/循环） - 工具集成策略（API 封装、认证管理、错误处理、限流降级） - 异常处理机制（重试、降级、告警、状态追踪） - 定时任务调度与监控

10.1 需求分析

10.1.1 任务自动化场景分类

任务自动化 Agent 根据应用场景可分为四类，每类有不同的设计重点：

| 场景类型 | 典型用例 | 设计重点 | 编排策略 | |---------|---------|---------|---------| | **工作流程自动化** | 收集数据→处理→生成报告→发送邮件 | 流程编排、状态传递 | 串行编排为主 | | **定时任务** | 每天 9 点抓取新闻、每周生成周报 | 调度精度、并发控制 | Cron 调度 + 分布式锁 | | **跨系统集成** | CRM 客户数据→邮件系统→发送营销邮件 | API 封装、认证管理 | 工具集成优先 | | **事件驱动任务** | 收到新订单→通知仓库→更新库存 | 响应速度、可靠性 | 事件队列 + 重试 |

核心差异分析： - **工作流程**重编排：需要清晰的流程定义和状态传递 - **定时任务**重调度：需要精确的时间控制和并发管理 - **跨系统集成**重封装：需要统一的工具接口和错误处理 - **事件驱动**重响应：需要<100ms 响应和 99.9% 可靠的事件处理

> **关键定义**：漫剧大纲自动生成属于**工作流程自动化**。**API**（应用程序接口）：不同系统间通信的标准接口。，需要清晰的流程定义（检索→分析→生成→格式化）和状态传递（每步输出作为下一步输入）。

10.1.2 核心挑战：任务分解

为什么任务分解是任务自动化的核心挑战?

复杂任务无法一步完成，需要分解为可执行的原子步骤。

```
问题根源：复杂任务无法一步完成
│
├── 分解原则：每个子任务有明确输入输出、可独立执行、可验证结果
│
├── 依赖关系：子任务之间可能有先后依赖（A 完成才能执行 B）或并行关系
│
└── 动态调整：执行中可能需要根据中间结果调整后续任务（条件分支）
```

分解原则： 1. **原子性**：每个子任务不可再分，有明确的完成标准 2. **独立性**：子任务可独立执行（有依赖关系的需明确依赖） 3. **可验证**：子任务结果可验证（输出格式、质量要求明确）

依赖关系分析： - **串行依赖**：B 需要 A 的输出（如「检索设定」→「分析创意」） - **并行关系**：A 和 B 可同时执行（如「生成章节 1」「生成章节 2」「生成章节 3」） - **条件分支**：根据 A 的结果选择 B 或 C（如「质量检查通过→发布，不通过→返工」）

> **常见误区**：分解过粗（子任务仍复杂难以执行）或过细（编排开销大）。建议分解到「一个 API 调用可完成」的粒度。

10.1.3 核心挑战：工具集成

为什么工具集成是任务自动化的核心挑战?

不同工具/系统有不同 API、认证方式、错误处理机制。

| 维度 | 挑战 | 解决方案 | |------|------|---------| | **API 封装** | 不同工具接口不一致 | 统一 Tool 接口（execute 方法） | | **认证管理** | API Key/OAuth/令牌刷新 | 安全存储 + 自动刷新 + 备用账户 | | **错误处理** | 不同工具错误格式不同 | 统一错误分类 + 错误传播机制 | | **状态管理** | 工具调用可能有状态（如文件 ID） | 工作记忆记录并传递状态 | | **版本兼容** | 工具 API 可能升级 | 版本管理 + 兼容处理 |

实现架构：

```
┌──────────────────────────────────────────────┐ │ 任务自动化 Agent │
├──────────────────────────────────────────────┤ │ ┌──────────────┐
┌──────────┐ │ │ 向量数据库 │ │ LLM API │ ← 工具层 │ │
└──────────┘ └──────────┘ │ │ ↑ ↑ │ │
┌──────────────────────────┐ │ │ 统一 Tool 接口 │ ← 封装层 │ │ │
execute(input) → output│ │ │ └──────────────────────────┘ │
└──────────────────────────────────────────────┘
```

> **最佳实践**：漫剧系统集成向量数据库（检索设定）、LLM API（生成内容）、文件系统（保存输出），统一封装为 Tool 接口，便于错误处理和监控。

10.1.4 本节小结

任务自动化分**工作流程/定时任务/跨系统集成/事件驱动**四类，核心挑战是**任务分解**（依赖关系、动态调整）和**工具集成**（API 封装、认证管理、错误处理）。

10.2 方案设计：任务分解与编排

总：任务分解用层级分解、LLM 辅助、模板化方法，编排方式有串行（依赖关系）、并行（独立任务）、条件分支（动态路径）、循环（重试/批量），需要明确终止条件。

10.2.1 任务分解方法

三种分解方法：

| 方法 | 说明 | 适用场景 | 示例 | |------|------|---------|------| | **层级分解** | 大任务→阶段→步骤→原子操作 | 复杂任务 | 漫剧生成：创作阶段→大纲步骤→章节原子操作 | | **LLM 辅助分解** | 用 LLM 分析任务描述输出子任务列表 | 模糊任务 | Prompt「将以下任务分解为可执行的子任务」 | | **模板化分解** | 常见任务类型有预定义分解模板 | 重复任务 | 「报告生成」= 数据收集→处理→撰写→美化 |

验证标准：每个子任务有明确的完成标准（输出格式、质量要求）。

层级分解示例（漫剧大纲生成）：

```
漫剧大纲生成
├── 阶段 1：检索
│   └── 步骤 1.1：检索历史设定（原子操作）
├── 阶段 2：分析
│   └── 步骤 2.1：分析创意需求（原子操作）
├── 阶段 3：生成
│   ├── 步骤 3.1：生成章节框架（原子操作）
│   └── 步骤 3.2：填充章节描述（原子操作，可并行）
└── 阶段 4：格式化
    └── 步骤 4.1：格式化输出（原子操作）
```

> **常见误区**：分解后子任务仍有歧义。需明确输入输出和完成标准。例如「生成章节描述」应明确为「输入：章节框架，输出：300-500 字章节描述，质量要求：符合角色设定」。

10.2.2 串行编排

定义：串行编排适用于子任务有严格先后依赖的场景（B 需要 A 的输出）。

实现方式： - 任务队列，按顺序执行 - 前一任务完成触发后一任务 - 前一任务的输出作为后一任务的输入（如文件路径、数据对象）

状态传递： `任务 A（输出：设定数据） → 任务 B（输入：设定数据） → 任务 C（输入：B 的输出）`

中断恢复： - 记录已完成任务 - 中断后从断点继续（跳过已执行步骤） - 状态持久化到数据库（避免内存丢失）

> **最佳实践**：漫剧大纲生成严格按顺序执行（检索→分析→生成→填充→格式化），每步依赖前一步输出，状态持久化支持中断恢复。

10.2.3 并行编排

定义：并行编排适用于子任务相互独立的场景（A 和 B 可同时执行，结果互不影响）。

实现方式： - 并发执行（多线程/多进程/异步） - 等待所有任务完成后汇总结果 - 资源限制（并发数、超时控制）

资源限制： | 限制类型 | 说明 | 建议值 | |---------|------|-------| | **并发数** | 同时执行的任务数 | 5-10（避免耗尽 API 配额） | | **超时控制** | 单个任务最大执行时间 | 30-60 秒（防止卡住） | | **重试次数** | 失败后重试次数 | 3 次（超过则判定失败） |

结果汇总：并行任务结果需要合并（如多个章节生成后合并为完整大纲）。

> **案例**：漫剧大纲的 5 个章节描述生成相互独立，用并行编排同时生成，速度提升 5 倍（从 5 分钟降至 1 分钟）。
>
> **常见误区**：盲目并行。有依赖关系的任务不能并行，需要分析依赖图。例如「生成章节框架」必须在「填充章节描述」之前，不能并行。

10.2.4 条件分支与循环

条件分支：根据中间结果选择不同路径。

实现方式： - 条件判断节点，根据条件表达式选择下一任务 - 条件表达式可以是质量评分、关键词匹配、布尔值等

示例（漫剧质量检查）： `生成大纲 → 质量检查 │ ├── 评分 > 80 → 发布 │ ├── 60 ≤ 评分 ≤ 80 → 返工修改 │ └── 评分 < 60 → 转人工审核`

循环处理： - **任务失败重试**：最多 N 次（固定 3 次） - **批量处理**：对每个项目执行相同任务（如生成 10 个章节） - **终止条件**：循环必须有明确终止条件（如重试 3 次仍失败则放弃）

> **最佳实践**：漫剧大纲生成后质量检查，评分>80 发布，60-80 返工修改，<60 转人工审核。返工修改最多 2 次，超过则转人工。

10.2.5 本节小结

任务分解用**层级/LLM 辅助/模板化**方法，编排方式有**串行**（依赖关系）、**并行**（独立任务）、**条件分支**（动态路径）、**循环**（重试/批量），需要明确**终止条件**。

10.3 方案设计：工具集成策略

总：工具集成需要 API 封装（统一接口）、认证管理（安全存储/轮换）、错误处理（分类/传播/恢复）、限流降级（等待/切换/熔断），确保任务稳定执行。

10.3.1 API 封装

定义：为不同工具定义统一接口，屏蔽底层差异。

统一接口设计： ```python class Tool: def execute(self, input: Dict) -> Dict: """执行工具调用""" pass

```
def validate_input(self, input: Dict) -> bool:
    """验证输入参数"""
    pass

def parse_output(self, raw_output: Any) -> Dict:
    """解析输出为标准格式"""
    pass
```

输入验证： - 检查输入参数格式、类型、范围 - 提前发现错误，避免无效调用

输出解析： - 将工具返回数据转换为标准格式（如统一为 Dict 或 Dataclass） - 处理工具返回的异常情况（如空值、错误码）

超时设置： - 为每个工具调用设置超时（固定 30 秒） - 避免卡住导致整个任务阻塞

> **最佳实践**：漫剧系统封装向量数据库、LLM API、文件系统为统一 Tool 接口，输入输出格式标准化，便于错误处理和监控。

10.3.2 认证管理

认证方式对比：

| 方式 | 说明 | 管理要点 | |------|------|---------| | **API Key** | 静态密钥 | 安全存储、定期轮换、权限最小化 | | **OAuth** | 用户授权获取访问令牌 | 令牌过期自动刷新、刷新令牌安全存储 | | **多账户** | 支持多个账户切换 | 负载均衡、认证失败时切换备用账户 |

安全存储： - ❌ 禁止硬编码在代码中 - ✅ 使用环境变量或密钥管理服务（如 AWS Secrets Manager） - ✅ 定期轮换（如每 90 天）

认证失败处理： - 检测认证失败（401/403 错误码） - 触发告警或切换备用账户 - 记录失败日志便于问题定位

> **最佳实践**：漫剧系统配置多个 LLM API Key，认证失败时自动切换备用 Key，避免任务中断。Key 存储在环境变量中，不提交到代码仓库。

10.3.3 错误处理

错误分类：

| 类型 | 示例 | 可恢复性 | 处理策略 | |------|------|---------|---------| | **网络错误** | 超时、连接失败 | ✅ 可恢复 | 重试 | | **API 错误（4xx）** | 400 参数错误、401 认证失败 | ❌ 不可恢复 | 记录错误，通知用户 | | **API 错误（5xx）** | 500 服务器错误、503 服务不可用 | ✅ 可恢复 | 重试 + 降级 | | **业务错误** | 参数错误、权限不足 | ❌ 不可恢复 | 记录错误，通知用户 |

错误传播： - 工具错误需要向上层任务传递 - 包含错误类型、错误信息、可恢复性 - 便于上层任务决策（重试/跳过/终止）

错误日志： - 记录错误详情（时间、工具、输入、错误信息） - 便于问题定位和复盘

> **案例**：漫剧系统调用 LLM API 超时，记录错误日志后重试，3 次失败后切换备用 API。

10.3.4 限流与降级

限流检测：- 识别限流错误（429 Too Many Requests）- 提取重试时间（Retry-After 头）

限流应对策略：

| 策略 | 说明 | 适用场景 | |------|------|---------| | **等待重试** | 等待 Retry-After 指定时间后重试 | 短期限流 | | **降低频率** | 降低调用频率（如从每秒 10 次降至 1 次） | 持续限流 | | **切换服务** | 切换到备用 Provider | 多 Provider 配置 |

降级策略：- **功能降级**：核心功能不可用时提供简化版本（如 LLM 不可用时用预定义模板）- **质量降级**：降低输出质量要求（如用轻量模型替代重量模型）- **用户通知**：降级时告知用户，管理预期

熔断机制：- 连续失败 N 次后熔断（暂停调用）- 一段时间后尝试恢复（如 5 分钟后）- 避免持续调用不可用服务

> **最佳实践**：漫剧系统 LLM API 触发限流，等待 60 秒后重试，3 次限流后切换到备用 Provider。GPT-4 不可用时降级用 GPT-3.5，告知用户「使用备用模型，生成速度可能较快但质量略低」。

10.3.5 本节小结

工具集成需要**API 封装**（统一接口）、**认证管理**（安全存储/轮换）、**错误处理**（分类/传播/恢复）、**限流降级**（等待/切换/熔断），确保任务稳定执行。

10.4 方案设计：异常处理机制

总：异常处理包括重试（指数退避/最大次数）、降级（功能/数据/质量）、告警（分级/收敛）、状态追踪（持久化/可查询），确保任务可恢复可监控。

10.4.1 重试机制

重试条件：可恢复错误（网络超时、限流、临时服务不可用）。

重试策略对比：

| 策略 | 说明 | 优点 | 缺点 | |------|------|------|------| | **固定间隔** | 每 5 秒重试 | 简单 | 可能与服务恢复时间不匹配 | | **指数退避** | 1s→2s→4s→8s | 逐步增加等待时间 | 实现稍复杂 | | **带抖动** | 指数退避 + 随机抖动 | 避免并发请求同时重试 | 实现最复杂 |

推荐策略：指数退避 + 抖动 `重试间隔 = base_delay × (2 ^ retry_count) + random_jitter 例如：1s → 2s → 4s → 8s（每次加 0-1s 随机抖动）`

最大重试次数：固定 3 次，超过则判定为失败。

重试日志：记录每次重试时间、错误信息，便于分析问题。

> **最佳实践**：漫剧系统 LLM API 调用超时，用指数退避重试（1s→2s→4s），3 次失败后切换备用 API。

10.4.2 降级策略

降级类型：

| 类型 | 说明 | 示例 | |------|------|------| | **功能降级** | 核心功能不可用时提供简化版本 | LLM 不可用时用预定义模板回复 | | **数据降级** | 部分数据缺失时用默认值或缓存数据 | 检索不到设定时用默认设定 | | **质量降级** | 降低输出质量要求 | 用 GPT-3.5 替代 GPT-4 |

降级触发条件： - 核心服务不可用（连续失败 N 次） - 限流无法解决（等待后仍限流） - 成本超预算（切换到轻量模型）

用户通知： - 降级时告知用户（「系统繁忙，提供简化服务」） - 管理用户预期，避免质量投诉

> **案例**：漫剧系统 GPT-4 不可用时降级用 GPT-3.5，告知用户「使用备用模型，生成速度可能较快但质量略低」。

10.4.3 告警机制

告警条件： - 连续失败 N 次（如 5 次） - 错误率超过阈值（如 10%） - 关键任务失败（如核心功能不可用） - 服务不可用（如 LLM API 宕机）

告警分级：

| 级别 | 说明 | 通知渠道 | 响应时间 | |------|------|---------|---------| | **P0（紧急）** | 服务中断 | 电话 + 短信 + 即时通讯 | 5 分钟内 | | **P1（高）** | 功能受损 | 短信 + 即时通讯 | 30 分钟内 | | **P2（中）** | 性能下降 | 即时通讯 | 2 小时内 | | **P3（低）** | 轻微问题 | 邮件 | 24 小时内 |

告警收敛： - 相同告警 N 分钟内只通知一次（如 30 分钟） - 避免告警风暴（同一问题重复通知）

> **最佳实践**：漫剧系统 LLM API 连续 5 次失败，触发 P1 告警发送邮件和钉钉通知给运维人员。相同告警 30 分钟内只通知一次。

10.4.4 任务状态追踪

状态定义:

| 状态 | 说明 | |------|------| | **待执行** | 任务已创建，等待执行 | | **执行中** | 任务正在执行 | | **已完成** | 任务成功完成 | | **失败** | 任务执行失败（超过最大重试次数） | | **已取消** | 任务被用户取消 |

状态记录: - 任务开始/结束时间 - 执行结果（成功/失败） - 错误信息（失败时） - 重试次数

状态查询: - 提供接口查询任务状态和进度 - 例如「任务 A 执行到第 3 步，共 5 步」

状态持久化: - 任务状态存入数据库 - 支持中断恢复和历史追溯 - ❌ 禁止只在内存中存储（中断后丢失）

> **最佳实践**：漫剧大纲生成任务记录状态「执行中（3/5 步）」，用户可随时查询进度。状态存入 **PostgreSQL**（关系型数据库）数据库，支持中断恢复。

10.4.5 本节小结

异常处理包括**重试**（指数退避/最大次数）、**降级**（功能/数据/质量）、**告警**（分级/收敛）、**状态追踪**（持久化/可查询），确保任务可恢复可监控。

10.5 定时任务与调度

总：调度策略有固定时间/间隔/Cron/依赖触发，并发控制用锁和幂等性防止重复执行，任务优先级支持插队和防饥饿，调度监控检测漏执行和补执行。

10.5.1 调度策略

调度方式对比:

| 方式 | 说明 | 示例 | 适用场景 | |------|------|------|---------| | **固定时间** | 每天/每周/每月固定时间执行 | 「每天 9:00」「每周一 10:00」 | 日报、周报 | | **固定间隔** | 每隔 N 分钟/小时执行 | 「每 30 分钟」「每 4 小时」 | 数据同步 | | **Cron 表达式**（定时任务配置语法） | 灵活定义调度时间 | 「0 9 * * 1-5」工作日每天 9 点 | 复杂调度 | | **依赖触发** | 前置任务完成后触发 | 「数据导入完成后触发分析任务」 | 工作流 |

Cron 表达式示例:
`0 10 * * * # 每天 10:00 执行 0 9 * * 1-5 # 工作日每天 9:00 执行 0 0 1 * * # 每月 1 日 0:00 执行 */30 * * * * # 每 30 分钟执行`

> **案例**：漫剧系统用 Cron「0 10 * * *」每天 10 点自动生成昨日章节草稿。
>
> **常见误区**：固定间隔不考虑执行时间。任务执行时间长可能导致重叠执行（如每 30 分钟执行一次，但任务需要 40 分钟）。解决方案：用分布式锁防止重叠。

10.5.2 并发控制

问题：多实例部署时，如何确保同一任务只执行一次?

解决方案：

| 方案 | 说明 | 适用场景 | |------|------|---------| | **重复执行检测** | 任务未完成时新调度触发，检测并跳过或排队 | 单实例 | | **分布式锁** | 多实例部署时用锁确保同一任务只执行一次 | 多实例 | | **幂等性设计**（重复执行结果一致） | 任务重复执行不会产生副作用 | 所有场景 |

分布式锁实现（**Redis**：内存数据库，支持分布式锁）：`获取锁 → 执行任务 → 释放锁 │ └─ 获取失败 → 跳过执行（其他实例在执行）`

锁超时释放： - 锁设置超时时间（如 5 分钟） - 超时自动释放，避免死锁 - 任务执行时间超过超时时需要续期

幂等性设计： - 用唯一 ID 去重（如任务 ID） - 重复执行时检查是否已执行过 - 已执行过则直接返回结果

> **最佳实践**：漫剧系统每天生成任务用 Redis 锁，确保多实例部署时只执行一次。锁超时 5 分钟，任务执行时间超过 5 分钟时续期。

10.5.3 任务优先级

优先级定义：

| 优先级 | 说明 | 示例 | |--------|------|------| | **P0（紧急）** | 立即执行，插队 | 作者紧急需求 | | **P1（高）** | 高优先级，优先执行 | 核心功能任务 | | **P2（中）** | 正常优先级 | 日常任务 | | **P3（低）** | 低优先级，空闲时执行 | 批量生成任务 |

调度策略： - 高优先级任务插队执行 - 低优先级任务等待 - 高优先级任务分配更多资源（如更多并发数）

饥饿预防： - 低优先级任务等待时间过长时提升优先级 - 例如等待超过 1 小时，P3 提升为 P2

> **案例**：漫剧系统 P0 任务（作者紧急需求）插队执行，P3 任务（批量生成）等待空闲时执行。P3 任务等待超过 1 小时自动提升为 P2。

10.5.4 调度监控

监控内容： - 执行记录：记录每次调度时间、执行结果、执行时长 - 漏执行检测：检测应该执行但未执行的任务（如服务宕机导致） - 延迟告警：任务执行时间偏离预期时告警 - 补执行机制：漏执行的任务在恢复后补执行

漏执行检测： - 记录预期执行时间 - 检测实际执行时间与预期时间的偏差 - 偏差超过阈值（如 1 小时）触发告警

补执行机制： - 服务恢复后检查漏执行的任务 - 补执行漏执行的任务 - 记录补执行原因

> **最佳实践**：漫剧系统监控每天生成任务，检测漏执行后补执行并告警通知运维。告警阈值：执行时间偏离预期超过 1 小时。

10.5.5 本节小结

调度策略有**固定时间/间隔/Cron/依赖触发**，并发控制用**锁和幂等性**防止重复执行，任务优先级支持**插队和防饥饿**，调度监控检测**漏执行和补执行**。

10.6 最佳实践与陷阱

10.6.1 最佳实践

| 实践 | 说明 | 案例 | |------|------|------| | **任务分解清晰** | 每个子任务有明确输入输出和完成标准 | 漫剧生成任务分解到原子操作 | | **工具封装统一** | 统一接口便于错误处理和监控 | 统一 Tool 接口（execute 方法） | | **异常处理完善** | 重试/降级/告警/状态追踪机制完整 | 指数退避重试 + 分级告警 | | **调度监控到位** | 漏执行检测、延迟告警、补执行机制 | Redis 锁 + 漏执行告警 | | **文档齐全** | 任务流程图、工具接口文档、异常处理手册 | 漫剧系统完整文档 |

10.6.2 常见陷阱

| 陷阱 | 后果 | 解决方案 | |------|------|---------| | **任务分解过粗** | 子任务仍复杂难以执行 | 进一步分解到原子操作 | | **错误处理缺失** | 任务失败无重试无降级 | 完善异常处理机制 | | **调度冲突** | 多实例重复执行 | 分布式锁和幂等性设计 | | **监控盲区** | 任务失败无告警 | 完善监控和告警机制 | | **资源耗尽** | 并发过多耗尽资源 | 并发数限制和资源监控 |

案例：漫剧系统初期无并发限制，批量生成时耗尽 API 配额，后增加并发数限制（最多 5 个并发）和配额监控（剩余额度<20% 告警）。

10.6.3 本节小结

最佳实践包括**任务分解清晰、工具封装统一、异常处理完善、调度监控到位、文档齐全**，常见陷阱是**分解过粗、错误处理缺失、调度冲突、监控盲区、资源耗尽**。

10.7 简单举例

案例设计

- 案例名称：漫剧大纲自动生成
- 涉及知识点：任务自动化 Agent 的任务分解、工具集成、异常处理、定时调度
- 案例目标：帮助理解如何设计一个自动化的漫剧大纲生成任务流程
- 案例内容要点：
- 场景描述：每天 10 点自动生成昨日章节草稿，包括设定检索、大纲生成、格式输出
- 技术应用：Cron 每天 10 点触发并用 Redis 锁防重复，任务分检索→生成→格式化三步，封装向量数据库和 LLM API 为 Tool 接口，指数退避重试 3 次失败后切换备用 API，状态存入 PostgreSQL
- 效果说明：自动化生成节省人工时间（从 30 分钟降至 1 分钟），异常处理确保任务稳定执行（成功率从 70% 提升至 95%），状态追踪支持进度查询
- 注意事项：不展开 Redis 锁的具体实现（见第 17 章）

知识来源: - LangChain Plan-and-Execute 官方文档 - https://python.langchain.com/docs/agents/plan_and_execute - AutoGen GroupChat 官方文档 - https://microsoft.github.io/autogen/docs/groupchat - ReAct 论文：Reasoning and Acting in Language Models (ICLR 2023) - https://arxiv.org/abs/2210.03629

修改记录: - v2.2 (2026-03-23): 量化指标 — 模糊表述改为具体数字（响应时间、重试次数、超时时间） - v2.1 (2026-03-23): 术语定义 — 首次出现必定义（API、Cron、Redis、幂等性、PostgreSQL） - v2.0 (2026-03-23): 文字润色 — 句子简化、删除重复、优化结构 - v1.1 (2026-03-22): 根据编辑统筹意见修改 — 规范知识来源格式（2-3 个权威来源） - v1.0 (2026-03-22): 初稿完成